

Natural Language Processing (NLP)

Regarding Course

Instructor: Sourav Kumar Dandapat(Sourav@iitp.ac.in)

Teaching Assistant:

- Srishti Gupta(srishti_2021cs38@iitp.ac.in)
- Shivani Gupta(shivani_2221cs18@iitp.ac.in)
- Rahul Kumar(rahul_2121cs10@iitp.ac.in)

Class Timing:

- Monday (4 pm-5pm)
- Tuesday (4 pm-5pm)
- Thursday (4 pm-5pm)

Course Page: 10.22.10.100/~sourav/nlp_2025/

Evaluation Plan:

- 30% class evaluation
- 30% Mid sem
- 40% End sem

Tentative Dates for Quizzes

- 2 quizzes before midsem (28/01, 18/02) [best of 2]
- 2 quizzes/presentation/projects after midsem [will be announced]

Text Book

- Speech and Language Processing (Daniel Jurafsky)

What is Natural Language Processing (NLP)

The process of computer analysis of input provided in a human language (natural language), and conversion of this input into a useful form of representation.

The field of NLP is primarily concerned with getting computers to perform useful and interesting tasks with human languages.

Forms of Natural Language

The input/output of a NLP system can be:

- **written text**
- **speech**

We will mostly concerned with written text (not speech).

To process written text, we need:

- **lexical, syntactic, semantic knowledge about the language**
- **discourse information, real world knowledge**

To process spoken language, we need everything required to process written text, plus the challenges of speech recognition and speech synthesis.

Components of NLP

Natural Language Understanding

- Mapping the given input in the natural language into a useful representation.
- Different level of analysis required:
morphological analysis,
syntactic analysis,
semantic analysis,
discourse analysis, ...

Natural Language Generation

- Producing output in the natural language from some internal representation.
- Different level of synthesis required:
deep planning (what to say),
syntactic generation

Why NL Understanding is hard?

Natural language is extremely rich in form and structure, and **very ambiguous**.

- How to represent meaning,

.

One input can mean many different things. Ambiguity can be at different levels.

- Lexical (word level) ambiguity -- different meanings of words
- Syntactic ambiguity -- different ways to parse the sentence
- Interpreting partial information -- how to interpret pronouns
- Contextual information -- context of the sentence may affect the meaning of that sentence.

Many input can mean the same thing.

Interaction among components of the input is not clear.

Knowledge of Language

Phonology – concerns how words are related to the sounds that realize them.

Morphology – concerns how words are constructed from more basic meaning units called morphemes. A morpheme is the primitive unit of meaning in a language.

Syntax – concerns how can be put together to form correct sentences and determines what structural role each word plays in the sentence and what phrases are subparts of other phrases.

Semantics – concerns what words mean and how these meaning combine in sentences to form sentence meaning. The study of context-independent meaning.

Pragmatics – concerns how sentences are used in different situations and how use affects the interpretation of the sentence.

Discourse – concerns how the immediately preceding sentences affect the interpretation of the next sentence. For example, interpreting pronouns and interpreting the temporal aspects of the information.

World Knowledge – includes general knowledge about the world. What each language user must know about the other's beliefs and goals.

Ambiguity

I made her duck.

- How many different interpretations does this sentence have?
- What are the reasons for the ambiguity?
- The categories of knowledge of language can be thought of as ambiguity resolving components.
- How can each ambiguous piece be resolved?
- Does speech input make the sentence even more ambiguous?

Some interpretations of : **I made her duck .**

1. I cooked *duck* for her.
2. I cooked *duck* belonging to her.
3. I created a toy duck which she owns.
4. I caused her to quickly lower her head or body.
5. I used magic and turned her into a *duck*.

Some NLP Applications

Machine Translation – Translation between two natural languages.

Information Retrieval – Web search (uni-lingual or multi-lingual).

Query Answering/Dialogue – Natural language interface with a database system, or a dialogue system.

Report Generation – Generation of reports such as weather reports.

Some Small Applications –

- Grammar Checking, Spell Checking, Spell Corrector

Brief History of NLP

1940s –1950s: Foundations

- Development of formal language theory (Chomsky, Backus, Naur, Kleene)
- Probabilities and information theory (Shannon)

1957 – 1970s:

- Use of formal grammars as basis for natural language processing (Chomsky, Kaplan)
- Use of logic and logic based programming (Minsky, Winograd, Colmerauer, Kay)

1970s – 1983:

- Probabilistic methods for early speech recognition (Jelinek, Mercer)
- Discourse modeling (Grosz, Sidner, Hobbs)

1983 – 1993:

- Finite state models (morphology) (Kaplan, Kay)

1993 – present:

- Strong integration of different techniques, different areas.

Regular Expression

- We'll begin with the most important tool for describing text patterns: the regular expression.
- Regular expressions can be used to specify strings we might want to extract from a document "I need X"
- For defining strings like \$199 or \$24.99 for extracting tables of prices from a document.

Text Normalization

We'll then turn to a set of tasks collectively called text normalization. Normalizing text means converting it to a more convenient, standard form.

- **Tokenization**: separating out or tokenizing words from running text
- **Lemmatization**: the task of determining that two words have the same root, despite their surface differences
- **Stemming** refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word.
- **Sentence segmentation**: breaking up a text into individual sentences, using cues like periods or exclamation points.
- We'll introduce a metric called **Edit Distance** that measures how similar two strings are based on the number of edits (insertions, deletions, substitutions) it takes to change one string into the other.

Regular Expressions

- One of the most useful tools for text processing in computer science has been the **regular expression** (often shortened to **regex**).
- It is a language for specifying text search strings.
- Formally, a regular expression is an algebraic notation for characterizing a set of strings.
- A regular expression search function will search through the corpus (a single document or a collection) , returning all texts that match the pattern.
- Regular expressions come in many variants. We'll be describing extended regular expressions

Regular expressions are used everywhere

- Part of every text processing task
 - Not a general NLP solution
 - But very useful as part of those systems (e.g., for pre-processing or text formatting)
- Necessary for data analysis of text data
- A widely used tool in industry and academics

Basic Regular Expression Patterns

If you like to search a string, let's say "Bunny", you can do that by `/Bunny/`.

Regular expressions are case sensitive; lower case `/s/` is distinct from upper case `/S/` (`/s/` matches a lower case `s` but not an upper case `S`).

This means that the pattern `/woodchucks/` will not match the string `Woodchucks`.

We can solve this problem with the use of the square braces `[and ]`.

The string of characters inside the braces specifies a **disjunction** of characters to match.

Regex	Example Patterns Matched
/woodchucks/	“interesting links to <u>woodchucks</u> and lemurs”
/a/	“Ma <u>r</u> y Ann stopped by Mona’s”
/!/	“You’ve left the burglar behind again <u>!</u> ” said Nori

Regex	Match	Example Patterns
/[wW]oodchuck/	Woodchuck or woodchuck	“ <u>Woodchuck</u> ”
/[abc]/	‘a’, ‘b’, <u>or ‘c’</u>	“In uo <u>m</u> ini, in solda <u>t</u> i”
/[1234567890]/	any digit	“plenty of <u>7</u> to 5”

Regular Expressions: Ranges

- The regular expression `/[1234567890]/` specifies any single digit.
- The regular expression `/[ABCDEFGHIJKLMNOPQRSTUVWXYZ]/` matches with any capital letter.
- In cases where there is a well-defined sequence associated with a set of characters, the brackets can be used with the dash (-) to specify any one character in a range.

Ranges using the dash:

Pattern	Matches	
<code>[A-Z]</code>	An upper case letter	<u>D</u> renched Blossoms
<code>[a-z]</code>	A lower case letter	<u>m</u> y beans were impatient
<code>[0-9]</code>	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expressions: Negation in Disjunction

Carat as first character in [] negates the list

- Note: Carat means negation only when it's first in []
- Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
<u>[^A-Z]</u>	Not an upper case letter	O <u>y</u> fn pripetchik
<u>[^Ss]</u>	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
<u>[^.]</u>	Not a period	<u>O</u> ur resident Djinn
<u>[e^]</u>	Either e or ^	Look up <u>^</u> now

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
/\d	[0-9]	Any digit	Fahre n eit <u>4</u> 51
/\D	[^0-9]	Any non-digit	<u>B</u> lue Moon
/\w	[a-zA-Z0-9_]	Any alphanumeric or _	<u>D</u> aiyu
/\W	[^\w]	Not <u>alphanumeric or _</u>	Look <u>!</u>
/\s	[\r\t\n\f]	Whitespace (space, tab)	Look <u>_</u> up
/\S	[^\s]	Not whitespace	<u>L</u> ook up

Regular Expressions: More Disjunction

Groundhog is another name for woodchuck!

The pipe symbol | for disjunction

Pattern	Matches
<code>groundhog woodchuck</code>	<code>woodchuck</code>
<code>yours mine</code>	<code>yours</code>
<code>a b c</code>	<code>= [abc]</code>
<code>[gG]roundhog [Ww]oodchuck</code>	<code>Woodchuck</code>



Wildcards, optionality, repetition: . ? * +

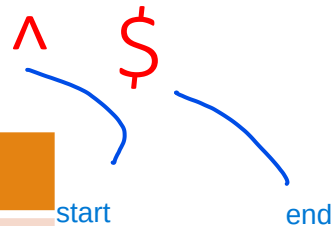
Pattern	Matches	Examples
beg.n <u> </u>	Any char	<u>begin</u> <u>begun</u> <u>beg3n</u> <u>beg n</u>
woodchucks? <u> </u>	Optional <u>s</u>	<u>woodchuck</u> <u>woodchucks</u>
to* <u> </u>	0 or more of <u>previous char</u>	<u>t</u> <u>to</u> <u>too</u> <u>tooo</u>
to+ <u> </u>	1 or more of <u>previous char</u>	<u>to</u> <u>too</u> <u>tooo</u> <u>toooo</u>



Stephen C Kleene

Kleene *, Kleene +

Regular Expressions: Anchors



Pattern	Matches
<code>/^The/</code>	Matches <u>a line starts with "The"</u>
<code>^[A-Z]</code>	<u>P</u> alo Alto
<code>^[^A-Za-z]</code>	<u>1</u> "Hello"
<code>\. \$</code>	The end <u>.</u>
<code><u>.\$</u></code>	The end <u>?</u> The end <u>!</u>
<code>/^The dog\. \$/</code>	Matches a line that contains only the phrase "The dog."

Regex	Match
<code>^</code>	start of line
<code>\$</code>	end of line
<code>\b</code>	word boundary
<code>\B</code>	non-word boundary

`\b` matches a word boundary, and `\B` matches a non word-boundary. Thus, `^bthe\b/` matches the word “the” but not the word “other”.

precedence

- How can we specify both guppy and guppies? We cannot simply say `/guppy|ies/`, because that would match only the strings guppy and ies.
- This is because sequences like guppy take precedence over the disjunction operator `|`.
- To make the disjunction operator apply only to a specific pattern, we need to use the parenthesis operators `(` and `)`.
- So the pattern `/gupp(y|ies)/` would specify that we meant the disjunction only to apply to the suffixes y and ies.

- Kleene* operator applies by default only to a single character, not to a whole sequence.
- Suppose we want to match repeated instances of a string.
- Perhaps we have a line that has column labels of the form Column 1
Column 2 Column 3.
- The expression `/Column_[0-9]+_*/` will not match any number of columns; instead, it will match a single column followed by any number of spaces!
- With the parentheses, we could write the expression `/(Column_[0-9]+_*)*/` to match the word Column, followed by a number and optional spaces, the whole pattern repeated zero or more times.

This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the operator precedence hierarchy for regular expressions.

Parenthesis

()

Counters

* + ? {}

Sequences and anchors

the ^my end\$

Disjunction

|

The iterative process of writing regex's

Find me all instances of the word “the” in a text.

the

Misses capitalized examples

[tT]he

Incorrectly returns other or Theology

 `\W[tT]he\W`

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

- 1. Not matching things that we should have matched
(The)

False negatives

- 2. Matching strings that we should not have matched
(there, then, other)

False positives

Characterizing work on NLP

In NLP we are always dealing with these kinds of errors.

Reducing the error rate for an application often involves two antagonistic efforts:

- Increasing coverage (or *recall*) (minimizing false negatives).
- Increasing accuracy (or *precision*) (minimizing false positives)

Regular expressions play a surprisingly large role

Widely used in both academics and industry

1. Part of most text processing tasks, even for big neural language model pipelines
 - including text formatting and pre-processing
2. Very useful for data analysis of any text data

Basic Text Processing

More Regular Expressions: Substitutions and ELIZA

Substitutions

Substitution in Python and UNIX commands:

```
s/regex1/pattern/
```

e.g.:

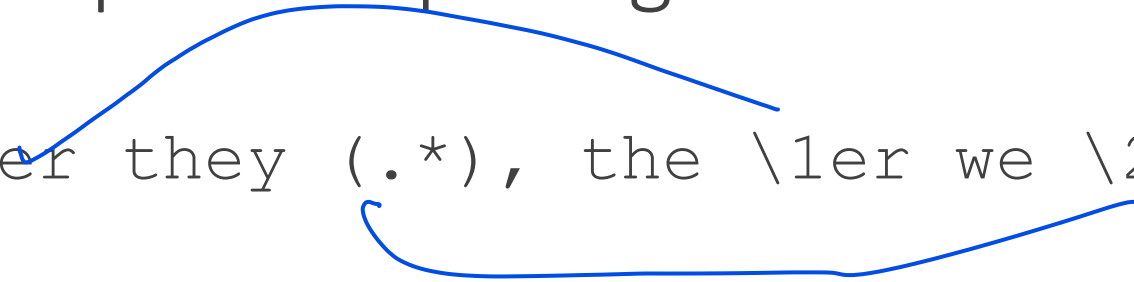
```
s/colour/color/
```

Capture Groups

- Say we want to put angles around all numbers:
the 35 boxes → *the <35> boxes*
- Use parens () to "capture" a pattern into a numbered register (1, 2, 3...)
- Use \1 to refer to the contents of the register
`s / ( [0-9] + ) / <\1> /`

Capture groups: multiple registers

`/the (.*)er they (.*) , the \1er we \2/`



Matches

*the **faster** they **ran**, the **faster** we **ran***

But not

*the **faster** they **ran**, the **faster** we ate*

But suppose we don't want to capture?

Parentheses have a double function: grouping terms, and capturing

Non-capturing groups: add a ? after paren:

```
/ (? : some | a few) (people | cats) like some \1 /
```

matches

- some cats like some cats

but not

- some cats like some some

Lookahead assertions

`/(?= pattern)` is true if pattern matches, but is **zero-width; doesn't advance character pointer**

`(?! pattern)` true if a pattern does not match

How to match, at the beginning of a line, any single word that doesn't start with "Volcano":

`/^(?!Volcano) [A-Za-z]+/`

Simple Application: ELIZA

Early NLP system that imitated a Rogerian psychotherapist

- Joseph Weizenbaum, 1966.

Uses pattern matching to match, e.g.,:

- "I need X"

and translates them into, e.g.

- "What would it mean to you if you got X?"

Simple Application: ELIZA

Men are all alike.

IN WHAT WAY

They're always bugging us about something or other.

CAN YOU THINK OF A SPECIFIC EXAMPLE

Well, my boyfriend made me come here.

YOUR BOYFRIEND MADE YOU COME HERE

He says I'm depressed much of the time.

I AM SORRY TO HEAR YOU ARE DEPRESSED

How ELIZA works

s/. * I'M (depressed|sad) . */I AM SORRY TO HEAR YOU ARE \1/
s/. * I AM (depressed|sad) . */WHY DO YOU THINK YOU ARE \1/
s/. * all . */IN WHAT WAY?/
s/. * always . */CAN YOU THINK OF A SPECIFIC EXAMPLE?/

How many words in a sentence?

"I do uh main- mainly business data processing"

- Fragments, filled pauses

"Seuss's **cat** in the hat is different from other **cats**!"

- **Lemma**: same stem, part of speech, rough word sense
 - **cat** and **cats** = same lemma
- **Wordform**: the full inflected surface form
 - **cat** and **cats** = different wordforms

How many words in a sentence?

they lay back on the San Francisco grass and looked at the stars
and their

Type: an element of the vocabulary.

Token: an instance of that type in running text.

How many?

- 15 tokens (or 14)
- 13 types (or 12) (or 11?)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, **$|V|$** is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^\beta$ where often $0.67 < \beta < 0.75$

i.e., vocabulary size grows with $>$ square root of the number of word tokens

	Tokens = N	Types = $ V $
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer(s),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Corpora vary along dimension like

- **Language:** 7097 languages in the world
- **Variety**, like African American English varieties.
 - AAE Twitter posts might include forms like "*iont*" (*I don't*)
- **Code switching**, e.g., Spanish/English, Hindi/English:
 - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:]
 - H/E: dost tha or ra- hega ... dont worry ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics:** writer's age, gender, ethnicity, SES

Corpus datasheets

Gebbru et al (2020), Bender and Friedman (2018)

Motivation:

- Why was the corpus collected?
- By whom?
- Who funded it?

Situation: In what situation was the text written?

Collection process: If it is a subsample how was it sampled? Was there consent? Pre-processing?

+Annotation process, language variety, demographics, etc.

Basic Text Processing

Word tokenization

Text Normalization

Every NLP task requires text normalization:

- 
1. Tokenizing (segmenting) words
 2. Normalizing word formats
 3. Segmenting sentences

Space-based tokenization

A very simple way to tokenize

- For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
- Segment off a token between instances of spaces

Unix tools for space-based tokenization

- The "tr" command
- Inspired by Ken Church's UNIX for Poets
- Given a text file, output the word tokens and their frequencies

Simple Tokenization in UNIX

(Inspired by Ken Church's UNIX for Poets.)

Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < shakes.txt
```

Change all non-alpha to newlines

```
| sort
```

Sort in alphabetical order

```
| uniq -c
```

Merge and count each type

```
1945 A
```

```
72 AARON
```

```
19 ABBESS
```

```
5 ABBOT
```

```
... ..
```

```
25 Aaron
```

```
6 Abate
```

```
1 Abates
```

```
5 Abbess
```

```
6 Abbey
```

```
3 Abbot
```

```
.... ..
```

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

More counting

Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'a-z' '\n' | sort | uniq -c
```

Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
```

Issues in Tokenization

Can't just blindly remove punctuation:

- m.p.h., Ph.D., AT&T, cap'n
- prices (\$45.55)
- dates (01/02/06)
- URLs (<http://www.stanford.edu>)
- hashtags (#nlproc)
- email addresses (someone@cs.colorado.edu)

Clitic: a word that doesn't stand on its own

- "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#)

When should multiword expressions (MWE) be words?

- [New York](#), [rock 'n' roll](#)

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...      ([A-Z]\.)+        # abbreviations, e.g. U.S.A.
...      | \w+(-\w+)*      # words with optional internal hyphens
...      | \$?\d+(\.\d+)?%? # currency and percentages, e.g. $12.40, 82%
...      | \.\.\.          # ellipsis
...      | [][.,;"'()?:-_'] # these are separate tokens; includes ], [
...      '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization in languages without spaces

Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?

Word tokenization in Chinese

Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.

How to do word tokenization in Chinese?

.

.

姚明进入总决赛 “Yao Ming reaches the finals”

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

7 characters? (don't use words at all):

姚 明 进 入 总 决 赛

Yao Ming enter enter overall decision game

Word tokenization / segmentation

So in Chinese it's common to just treat each character (zi) as a token.

- So the **segmentation** step is very simple

In other languages (like Thai and Japanese), more complex word segmentation is required.

- The standard algorithms are neural sequence models trained by supervised machine learning.

Basic Text Processing

Byte Pair Encoding

Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

Use the data to tell us how to tokenize.

Subword tokenization (because tokens can be parts of words as well as whole words)

Subword tokenization

Three common algorithms:

- **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
- **Unigram language modeling tokenization** (Kudo, 2018)
- **WordPiece** (Schuster and Nakajima, 2012)

All have 2 parts:

- A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
- A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,..., a, b, c, d....}

Repeat:

- Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
- Add a new merged symbol 'AB' to the vocabulary
- Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until k merges have been done.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '___' before space in training corpus

Next, separate into letters.

BPE token learner

Original (very fascinating 🤔) corpus:

low low low low low lowest lowest newer newer newer
newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

vocabulary

—, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Merge **e r** to **er**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge **er _** to **er_**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Merge **n e** to **ne**

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

BPE

The next merges are:

Merge	Current Vocabulary
(ne, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new
(l, o)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo
(lo, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low
(new, er—)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—
(low, —)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—, low—

BPE token **segmenter** algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test frequencies don't play a role)

So: merge every **e r** to **er**, then merge **er _** to **er_**, etc.

Result:

- Test set "n e w e r _" would be tokenized as a full word
- Test set "l o w e r _" would be two tokens: "low er_"

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

Basic Text Processing

Word Normalization and other issues

Word Normalization

Putting words/tokens in a standard format

- U.S.A. or USA
- uhhuh or uh-huh
- Fed or fed

Case folding

Applications like IR: reduce all letters to lower case

- Since users tend to use lower case
- Possible exception: upper case in mid-sentence?
 - e.g., *General Motors*
 - *Fed* vs. *fed*
 - *SAIL* vs. *sail*

For sentiment analysis, MT, Information extraction

- Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root

• = dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

Lemmatization is done by Morphological Parsing

Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*

Porter Stemmer

Based on a series of rewrite rules run in series

- A cascade, in which output of each pass fed to next pass

Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ϵ if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- **Uygarlastiramadiklarimizdanmissinizcasina**
- `(behaving) as if you are among those whom we could not civilize’
- **Uygar** `civilized’ + **las** `become’
 - + **tir** `cause’ + **ama** `not able’
 - + **dik** `past’ + **lar** `plural’
 - + **imiz** `p1pl’ + **dan** `abl’
 - + **mis** `past’ + **siniz** `2pl’ + **casina** `as if’

Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.